Fundamentele programării 3.

Sorin Milutinovici
sorin.milutinovici@prof.utm.ro

Pointeri

Referințe

Referințele sunt valori care permit accesarea indirectă a altor date. Alte date pot fi:

- Conținutul altor variabile
- Conținutul unui fișier stocat pe un harddisk
- Conținutul unei anumite zone de memorie
- Conținutul unei alte structuri de date

O referință este diferită de datele respective. De cele mai multe ori, o referință reprezintă **o adresă fizică a unei poziții de memorie** dar sunt situații în care acestea sunt reprezentate altfel. De exemplu, adresa unei pagini internet se numește tot referință.

În sens strict, în programare, referința reprezintă o adresă de memorie care este astfel definită încât compilatorul să poată accesa corect datele respective.

Operația de accesare a datelor din spatele referinței se numește dereferențiere și se face diferit în funcție de limbajul de programare ales.

În C, pentru a accesa datele care se găsesc “în spatele” unei adrese de memorie care se constituie în referință, această adresă trebuie stocată într-un tip special de variabilă numit pointer.

Pointerii se declară împreună cu un tip de variabilă. Astfel, compilatorul știe cât din memorie trebuie să citească în momentul în care este nevoie de dereferențiere.

Variabile și referințe

variables.c

```c
#include <stdio.h>

int main()
{
int a = 100;
printf ("adresa a: %p, lungime %d\n", &a, sizeof(a));
char d = 'b';
printf ("adresa d: %p, lungime %d\n", &d, sizeof(d));
short m = 21;
printf ("adresa m: %p, lungime %d\n", &m, sizeof(m));
long v = 100;
printf ("adresa v: %p, lungime %d\n", &v, sizeof(v));
}
```

https://onlinegdb.com/SJ-dgMiwV

Prima rulare a programului

```
adresa a: 0x7ffe5fce6084, lungime 4
adresa d: 0x7ffe5fce6081, lungime 1
adresa m: 0x7ffe5fce6082, lungime 2
adresa v: 0x7ffe5fce6088, lungime 8
```

A doua rulare a programului

```
adresa a: 0x7ffe5fce6084, lungime 4
adresa d: 0x7ffe5fce6081, lungime 1
adresa m: 0x7ffe5fce6082, lungime 2
adresa v: 0x7ffe5fce6088, lungime 8
```

A treia rulare a programului

```
adresa a: 0x7ffcabb2d1b4, lungime 4
adresa d: 0x7ffcabb2d1b1, lungime 1
adresa m: 0x7ffcabb2d1b2, lungime 2
adresa v: 0x7ffcabb2d1b8, lungime 8
```

De ce sunt adrese diferite?

1. Nici un sistem de operare nu face eforturi pentru a menține baza stack-ului la aceeași adresă.
2. ASLR (address space layout randomization), funcțional în majoritatea sistemelor de operare, “amestecă” cu bună știință adresele pentru a crește securitatea sistemelor.

Consecință: nu putem utiliza adresele de memorie direct în programele noastre. De aceea avem nevoie de un mecanism care să “reînnoiască” adresele de care avem nevoie la fiecare rulare.

Referințe și pointeri

```
int a = 100;
printf ("adresa a: %p\n", &a);
```

&a se numește “**referință a variabilei a**” și conține adresa de memorie a variabilei a

Putem stoca această adresă de memorie într-un tip special de variabilă, numită “pointer”. Pointer-ul se definește cu ajutorul caracterului * și **trebuie să includă tipul variabilei** către care “punctează”.

```
int *pa = &a;
```

pointer1.c

```
#include <stdio.h>

int main()
{
  int a = 100;
  printf ("a = %d\n", a);
  printf ("adresa a: %p\n", &a);

  int *pa = &a;
  printf("*pa = %p\n", pa) ;
  *pa = 120;
  printf ("a = %d\n", a);
}
```

https://onlinegdb.com/Bkbw-h2Lu

Rezultatul

```
a = 100
adresa a: 0x7ffefdb4f104

*pa = 0x7ffefdb4f104

a = 120
```

Pointer către char

Presupunem ca o adresa de memorie se poate stoca in 8 biti !!

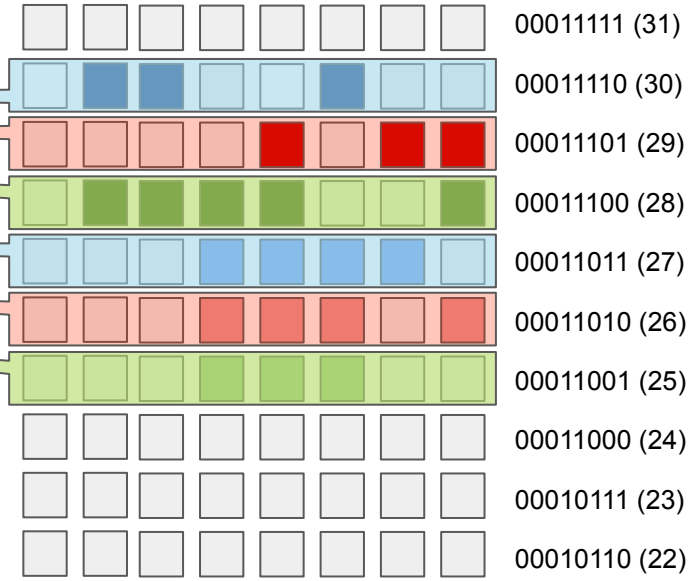
pointer_char.c

```c
#include <stdio.h>

int main()
{
char a = 100;
char b = 11;
char c = 121;

char *ap = &a;
char *bp = &b;
char *cp = &c;

printf ("adresa a: %p\n", &a);
printf ("adresa b: %p\n", &b);
printf ("adresa c: %p\n", &c);
printf("------------\n");

printf ("valoarea este: %d\n",*ap);
printf ("valoarea este: %d\n",*bp);
printf ("valoarea este: %d\n",*cp);
printf("------------\n");
ap++;
printf ("valoarea este: %d\n",*ap);
ap++;
printf ("valoarea este: %d\n",*ap);
}
```

https://onlinegdb.com/Ws62KqRwLS

Pointerul “sare” din octet în octet când este incrementat.

Conținutul memoriei (adrese 8 biti)



01100100 = 100 (a)

00001011 = 11 (b)

01111001 = 121 (c)

00011110 (adresa a)

00011101 (adresa b)

00011100 (adresa c)

```
adresa a: 0x7fffd2c0c425
adresa d: 0x7fffd2c0c426
adresa d: 0x7fffd2c0c427
------------
valoarea este: 100
valoarea este: 11
valoarea este: 121
------------
valoarea este: 11
valoarea este: 121
```

Diferență de 1 octet

Pointer către int

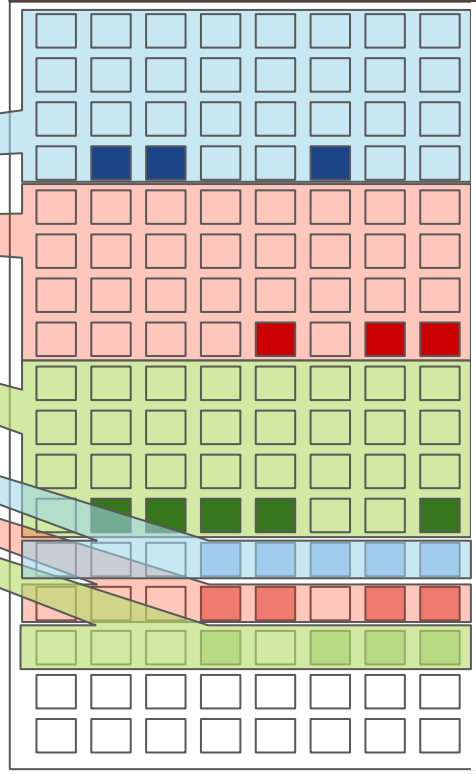
pointer_int.c

```c
#include <stdio.h>

int main()
{
int a = 100;
int b = 11;
int c = 121;

int *ap = &a;
int *bp = &b;
int *cp = &c;

printf ("adresa a: %p\n", &a);
printf ("adresa b: %p\n", &b);
printf ("adresa c: %p\n", &c);
printf("------------\n");

printf ("valoarea este: %d\n",*ap);
printf ("valoarea este: %d\n",*bp);
printf ("valoarea este: %d\n",*cp);
printf("------------\n");
ap++;
printf ("valoarea este: %d\n",*ap);
ap++;
printf ("valoarea este: %d\n",*ap);
}
```

https://onlinegdb.com/tpH-v61Mp

Pointerul “sare” corect din 4 în 4 octeți când este incrementat.

Conținutul memoriei (adrese 8 biti)



00011111 (31)	00000000
00011110 (30)	00000000
00011101 (29)	00000000
00011100 (28)	01100100 = 100 (a)
00011011 (27)	00000000
00011010 (26)	00000000
00011001 (25)	00000000
00011000 (24)	00001011 = 11 (b)
00010111 (23)	00000000
00010110 (22)	00000000
00010101 (21)	00000000
00010100 (20)	01111001 = 121 (c)
00010011 (19)	00011111 (adresa a)
00010010 (18)	00011011 (adresa b)
00010001 (17)	00010111 (adresa c)
00010000 (16)	
00001111 (15)	

```
adresa a: 0x7ffda8a93ccc
adresa d: 0x7ffda8a93cd0
adresa d: 0x7ffda8a93cd4
------------
valoarea este: 100
valoarea este: 11
valoarea este: 121
------------
valoarea este: 11
valoarea este: 121
```

Diferență de 4 octeți

Presupunem ca o adresa de memorie se poate stoca in 8 biti !!

Ce este de fapt un pointer?

Un pointer este un tip special de variabilă care stochează adresa de memorie a unei alte variabile împreună cu dimensiunea acesteia.

Un pointer știe exact **poziția la care începe stocarea unei variabile în memorie**

Un pointer știe exact **numărul de octeți utilizat la stocarea variabilelor.** Aceasta este posibil pentru că definirea pointerilor se face împreună cu tipul de variabilă către care acesta trimite.

Dimensiunea propriu-zisă a unui pointer (a variabilei de tip pointer) este aceeași de fiecare dată, indiferent de tipul de variabilă către care trimite.

Un pointer poate fi incrementat sau decrementat, acesta “mutându-se” de la începutul unei variabile la începutul următoarei (sau a celei precedente). Aceasta se numește “**aritmetica pointerilor**”.

Un pointer poate fi utilizat pentru obținerea valorii variabilei către care “punctează”. Această operație se numește **dereferențiere**.

Cantitatea de memorie in care se va stoca rezultatul dereferentierii este stabilita la compilare

Rolurile asteriscului și ampersandului

```
int a = 1021;

int *aptr;

int b = *aptr;

int c = x * y

aptr = &a;

c = x&y

if (x > 0 && y > 0)
```

*	Specificator pentru declararea unei variabile de tip pointer
*	Operator de dereferențiere
*	Operator pentru operația de înmulțire
&	Specificator al referinței
&	Operație binară AND
&&	Operație logică AND

Atenție la pointeri

Atentionare	Mozilla 1.0 1998 (C++) 540613 linii de cod	Apache1.3 - 98 (C) 48393 linii de cod
Using uninitialized memory	26.14	45.00
Dereferencing uninitialized pointer	1.73	0.00
Dereferencing NULL pointer	58.93	50.00
Dereferencing invalid pointer	0.00	5.00
Dereferencing pointer to freed memory	1.98	0.00
Leaking memory	9.75	0.00
Leaking a resource (such as a file)	0.09	0.00
Returning pointer to local stack variable	0.52	0.00
Returning pointer to freed memory	0.09	0.00
Illegal value passed to function	0.43	0.00
Divide by zero	0.34	0.00

A static analyzer for finding dynamic programming errors
By:Bush, WR (Bush, WR); Pincus, JD (Pincus, JD); Sielaff, DJ (Sielaff, DJ)
SOFTWARE-PRACTICE & EXPERIENCE
Volume: 30, Issue: 7, Pages: 775-802

Cu culoarea roșie sunt erori legate de pointere. Valorile sunt procentuale.

https://www.whitehouse.gov/oncd/briefing-room/2024/02/26/memory-safety-fact-sheet/

https://media.defense.gov/2022/Nov/10/2003112742/-1/-1/0/CSI_SOFTWARE_MEMORY_SAFETY.PDF

National Security Agency | Cybersecurity Information Sheet

Software Memory Safety

Executive summary

Modern society relies heavily on software-based automation, implicitly trusting developers to write software that operates in the expected way and cannot be compromised for malicious purposes. While developers often perform rigorous testing to prepare the logic in software for surprising conditions, exploitable software vulnerabilities are still frequently based on memory issues. Examples include overflowing a memory buffer and leveraging issues with how software allocates and de-allocates memory. Microsoft® revealed at a conference in 2019 that from 2006 to 2018 70 percent of their vulnerabilities were due to memory safety issues. [1] Google® also found a similar percentage of memory safety vulnerabilities over several years in Chrome®. [2] Malicious cyber actors can exploit these vulnerabilities for remote code execution or other adverse effects, which can often compromise a device and be the first step in large-scale network intrusions.

Examples of memory safe language include C#, Go, Java®, Ruby™, Rust®, and Swift®.

Singurul cu adevarat in discuție este Rust (pentru programarea de sistem) - celelalte au nevoie de mediu de execuție

2004 - Memory safe C compiler (implementat în OCAML)

http://www.seclab.cs.sunysb.edu/mscc/

2009 - Fail Safe C Compiler

https://staff.aist.go.jp/y.oiwa/FailSafeC/index-en.html

RUST este mai sigur datorită unui sistem de proprietate a variabilelor care este însoțit de un compilator care îl verifică.

NULL pointer

```
int *aptr = NULL;
```

Pointerul NULL este un pointer care trimite către adresa de memorie 0. În majoritatea cazurilor, aceasta adresă este goală.

Încercarea de dereferențiere a unui pointer de tip NULL va duce la o eroare de tip segmentation fault. Dar, un pointer de tip NULL poate fi testat și în majoritatea compilatoarelor, inițializarea pointerilor cu NULL și testarea lor înainte de orice tip de acces reprezintă un mod eficient de a evita erorile.

pointer generic (void pointer)

Un pointer generic este un pointer al cărui tip nu este cunoscut. Este capabil să primească orice fel de adresă fără erori (evident, dereferențierea nu e posibilă - pointerii generici **nu pot fi dereferențiați**). Practic, un pointer generic nu conține informații despre mărimea variabilei către care trimite. Programatorul va trebui să știe care este dimensiunea memoriei disponibile.

Sunt utilizați de exemplu în funcțiile de alocare a memoriei: acestea trebuie să returneze un pointer (altfel memoria alocată nu va putea fi accesată):

```
exemplu: int * newArr=(int*)malloc(size*sizeof(int));
```

constructia (int*) inaintea funcției malloc convertește pointerul returnat de malloc într-un pointer de tip int.
În multe situații se poate scrie și: int * `newArr = malloc(size*sizeof(int));` fără conversie: multe compilatoare vor converti automat pointerii generici la tipul corect. Utilizarea conversiei în această situație are avantaje și dezavantaje:

https://en.wikipedia.org/wiki/C_dynamic_memory_allocation#Advantages_to_casting

Funcții “impure”

O funcție “pură” are următoarele proprietăți:

- Valoarea returnată este de fiecare dată aceeași dacă argumentele sunt aceleași
- Execuția funcției nu are efecte secundare (modificarea altor date cu excepția valorii returnate)

https://onlinegdb.com/SywgbLaUO

```c
#include <stdio.h>

void smfunc(int a) {
   a = 121;
   printf ("In interior: %d\n", a);
}

int main()
{
   int a = 100;
   printf("Inainte: %d\n", a);
   smfunc(a);
   printf("Dupa: %d\n", a);
}
```

Funcția primește ca argument o valoare

```
Inainte: 100
In interior: 121
Dupa: 100
```

impure_nopointer.c

https://onlinegdb.com/BJP-GITU_

```c
#include <stdio.h>

void smfunc(int * a) {
   *a = 121;
   printf ("In interior: %d\n", *a);
}

int main()
{
   int a = 100;
   printf("Inainte: %d\n", a);
   smfunc(&a);
   printf("Dupa: %d\n", a);
}
```

Funcția primește ca argument un pointer

```
Inainte: 100
In interior: 121
Dupa: 121
```

impure_pointer.c

Funcții “impure” - swap

https://onlinegdb.com/H1NMuclw_

```c
#include <stdio.h>

void swap(int *v1,int *v2)
{
   int t;
    t   = *v1;
   *v1   = *v2;
   *v2   =  t;
}
int main()
{
   int a = 100, b=121;

   printf("Inainte: a = %d, b= %d\n", a,b);
   swap(&a, &b);
   printf("Dupa: a = %d, b= %d\n", a,b);
}
```

swap.c

```
Inainte: a = 100, b= 121
Dupa: a = 121, b= 100
```

Funcția primește pointeri către variabilele definite în main.

t este o variabilă temporară folosită pentru a stoca una dintre valori

Se stochează valoarea către care trimite *v1 în t

Se stochează valoarea către care trimite *v2 în *v1 (adresa lui v1)

Se stochează valoarea din t în *v2 (spațiul de memorie al lui v2)

Funcția a manevrat direct variabile definite și stocate în frame-ul funcției părinte, cu ajutorul referințelor și pointerilor.

Pointeri și matrici

array_decay1.c

```c
#include <stdio.h>

int main () {
   int n[] = { 5, 20, 29, 32, 63};

   int vsize = sizeof(n)/sizeof(n[0]);

   printf ("vsize: %d\n", vsize );
}
```

https://onlinegdb.com/HJtE2GC8d

```
vsize: 5
```

La definirea unui tablou, se poate obține dimensiunea acestuia în octeți. Folosind construcția:

```c
sizeof(n)/sizeof(n[0]
```

se poate obține numărul de elemente din tablou.

La trimiterea unui tablou ca argument al unei funcții, acesta pierde informația dimensională pentru că este automat transformat într-un pointer constant.

array_decay2.c

```c
#include <stdio.h>

void functie(int arr[]) {
   int vsize = sizeof(arr)/sizeof(arr[0]);
   printf("vsize in functie, %d\n", vsize);
}

int main () {
   int n[] = { 5, 20, 29, 32, 63 };

   int vsize = sizeof(n)/sizeof(n[0]);

   printf ("vsize in main: %d\n", vsize );

   functie(n);
}
```

https://onlinegdb.com/ryNg9qePu

```
main.c:4:23: warning: ‘sizeof’ on array
function parameter ‘arr’ will return size of
‘int *’ [-Wsizeof-array-argument]
main.c:3:18: note: declared here
vsize in main: 5
vsize in functie, 2
```

Acest comportament se numește “array to pointer decay”. Matricile in C sunt ne-atribuibile (nu putem scrie int d[] = n, ci trebuie sa scriem int * d = n, ceea ce va copia în d doar pointerul “decăzut” al matricii n

Pointeri și matrici

```
int arr[4] = {2,23,5,11};
int *p = arr;
```

	Adresa HEX	Adresa zecimal
2	0x7ffd65f990e0	140726314307808
23	0x7ffd65f990e4	140726314307812
5	0x7ffd65f990e8	140726314307816
11	0x7ffd65f990ec	140726314307820

Elementele array-ului pot fi accesate prin aritmetica pointerului p.

chiar și “arr” poate fi tratat ca un pointer

pointer_array.c

```
printf("adresele primului membru: \n");
printf("arr        este: %p\n", arr);
printf("&(arr[0]) este: %p\n", &(arr[0]));
printf("p          este: %p\n\n", p);

printf("adresele celui de-al doilea membru: \n");
printf("arr+1      este: %p\n", arr+1);
printf("&(arr[1]) este: %p\n", &(arr[1]));
printf("p + 1      este: %p\n\n", p+1);

printf("adresele celui de-al treilea membru: \n");
printf("arr+2      este: %p\n", arr+2);
printf("&(arr[2]) este: %p\n", &(arr[2]));
printf("arr        este: %p\n\n", p+2);

printf("adresele celui de-al patrulea membru: \n");
printf("arr+3      este: %p\n", arr+3);
printf("&(arr[3]) este: %p\n", &(arr[3]));
printf("p + 3      este: %p\n\n", p+3);
```

```
adresele primului membru:
arr        este: 0x7ffd65f990e0
&(arr[0]) este: 0x7ffd65f990e0
p          este: 0x7ffd65f990e0

adresele celui de-al doilea membru:
arr+1      este: 0x7ffd65f990e4
&(arr[1]) este: 0x7ffd65f990e4
p + 1      este: 0x7ffd65f990e4

adresele celui de-al treilea membru:
arr+2      este: 0x7ffd65f990e8
&(arr[1]) este: 0x7ffd65f990e8
arr        este: 0x7ffd65f990e8

adresele celui de-al patrulea
membru:
arr+3      este: 0x7ffd65f990ec
&(arr[3]) este: 0x7ffd65f990ec
p + 3      este: 0x7ffd65f990ec
```

https://onlinegdb.com/SkBT1kzPd

Pointeri și matrici

```
int arr[4] = {2,23,5,11};
int *p = arr;
```

Elementele array-ului pot fi accesate prin aritmetica pointerului p.

chiar și "arr" poate fi tratat ca un pointer

	Adresa HEX	Adresa zecimal
2	0x7ffd65f990e0	140726314307808
23	0x7ffd65f990e4	140726314307812
5	0x7ffd65f990e8	140726314307816
11	0x7ffd65f990ec	140726314307820

Adunarea cu 1 în cazul unui pointer de tip int (4 octeți) va duce la următoarea adresă de număr întreg. Adică, va aduna la adresa curentă 4 octeți.

Adunarea cu 1 în cazul oricărui pointer va duce către adresa următoarei variabile reale, adunând fix atâția octeți cât sunt necesari în funcție de tipul pointerului.

Astfel, în cazul unui pointer de tip char, adunarea cu 1 va adăuga un octet la adresă, în cazul unui short va adăuga doi octeți, în cazul unui double 8 octeți (dependent, evident de arhitectura și de dimensiunea tipurilor pe sistemul pe care se compilează programul

pointer_array1.c

https://onlinegdb.com/SylW0qlDO

```
adresele celui de-al doilea membru:

arr+1      este: 0x7ffd65f990e4
&(arr[1])  este: 0x7ffd65f990e4
p + 1      este: 0x7ffd65f990e4
```

p și arr sunt pointeri de tip int, deci arr + 1 este același lucru cu p + 1.

în cazul folosirii parantezelor pătrate, compilatorul face dereferențierea automata, arr[1] va fi 23 iar pentru a obține adresa este nevoie de operatorul &

Pointeri și matrici - accesul la valori

```
int arr[4] = {2,23,5,11};
int *p = arr;
```

Elementele array-ului pot fi accesate prin aritmetica pointerului p.

chiar și “arr” poate fi tratat ca un pointer

	Adresa HEX	Adresa zecimal
2	0x7ffd65f990e0	140726314307808
23	0x7ffd65f990e4	140726314307812
5	0x7ffd65f990e8	140726314307816
11	0x7ffd65f990ec	140726314307820

pointer_array.c

```
printf("--------------valorile---------- \n");
printf("valorile primului membru: \n");
printf("arr[0]     este: %d\n", arr[0]);
printf("*arr       este: %d\n", *arr);
printf("*p         este: %d\n", *p);

printf("valorile celui de-al doilea membru: \n");
printf("arr[1]     este: %d\n", arr[1]);
printf("*(arr+1)   este: %d\n", *(arr+1));
printf("*(p+1)     este: %d\n", *(p+1));

printf("valorile celui de-al treilea membru: \n");
printf("arr[2]     este: %d\n", arr[2]);
printf("*(arr+2)   este: %d\n", *(arr+2));
printf("*(p+2)     este: %d\n", *(p+2));

printf("valorile celui de-al patrulea membru: \n");
printf("arr[3]     este: %d\n", arr[3]);
printf("*(arr+3)   este: %d\n", *(arr+3));
printf("*(p+3)     este: %d\n", *(p+3));
```

https://onlinegdb.com/SkBT1kzPd

```
--------------valorile----------
valorile primului membru:
arr[0]     este: 2
*arr       este: 2
*p         este: 2

valorile celui de-al doilea membru:
arr[1]     este: 23
*(arr+1)   este: 23
*(p+1)     este: 23

valorile celui de-al treilea membru:
arr[1]     este: 5
*(arr+1)   este: 5
*(p+1)     este: 5

valorile celui de-al patrulea membru:
arr[1]     este: 11
*(arr+1)   este: 11
*(p+1)     este: 11
```

Returnarea unei matrici dintr-o funcție

insertion_copy.c

```c
#include <stdio.h>
#include <stdlib.h>

int main()

{
   int data[] = {12,23,5,7,2,9,11,13,9};
   int *sorted = NULL;
   int num = sizeof(data)/sizeof(data[0]);

   printf("Inainte de sort:\n");
   printstr(data,num);
   sorted = insertion(data, num);
   printf("Dupa sort :\n");
   printstr(data,num);
   printf("Sorted :\n");
   printstr(sorted,num);
   return 0;
}
```

https://onlinegdb.com/SygL1OwpQ

Se returnează un pointer către primul element al matricii, fără informație dimensională

```c
int * insertion(int data[], int size)
{
   int *newArr;
   int i,temp,j;
   newArr=(int*)malloc(size*sizeof(int));
   for (i = 0 ; i <= size - 1; i++)
   {
       newArr[i] = data[i];
   }
   for (i = 0 ; i <= size - 1; i++)
   {
     j = i;
    while ( j > 0 && newArr[j] < newArr[j-1])
       {
           temp = newArr[j];
           newArr[j] = newArr[j-1];
           newArr[j-1] = temp;
           j--;
       }
    }
    return newArr;
}
```

```c
void printstr(int data[], int size)
{
    int i;
    for (i = 0; i <= size - 1; i++)
    {
          printf("%d\n", data[i]);
    }
}
```

Returnarea unei matrici dintr-o funcție

insertion_copy.c

```c
#include <stdio.h>
#include <stdlib.h>

int main()

{
    int data[] = {12,23,5,7,2,9,11,13,9};
    int *sorted = NULL;
    int num = sizeof(data)/sizeof(data[0]);

    printf("Inainte de sort:\n");
    printstr(data,num);
    sorted = insertion(data, num);
    printf("Dupa sort :\n");
    printstr(data,num);
    printf("Sorted :\n");
    printstr(sorted,num);
    return 0;
}
```

Declarăm un pointer NULL care va primi adresa matricii sortate

Asteptam ceea ce returneaza functia intr-un pointer.

https://onlinegdb.com/SygL1OwpQ

```c
int * insertion(int data[], int size)
{
    int *newArr;
    int i,temp,j;
    newArr=(int*)malloc(size*sizeof(int));
    for (i = 1 ; i <= size - 1; i++)
    {
        newArr[i] = data[i];
    }
    for (i = 1 ; i <= size - 1; i++)
    {
      j = i;
     while ( j > 0 && newArr[j] < newArr[j-1])
        {
            temp = newArr[j];
            newArr[j] = newArr[j-1];
            newArr[j-1] = temp;
            j--;
        }
     }
     return newArr;
}
```

Declarăm un pointer NULL care va primi adresa spațiului de memorie alocat

Acesta NU e pointer! E înmulțire

Returnăm pointerul

```c
void printstr(int data[], int size)
{
    int i;
    for (i = 0; i <= size - 1; i++)
    {
        printf("%d\n", data[i]);
    }
}
```

Dereferențiere multiplă

indirection.c

```c
#include <stdio.h>

int main()
{
   int a = 23;
   int *p = &a;
   int **pp = &p;
   int ***ppp = &pp;

....

   printf
("---Dereferentiere---\n");
   printf ("a: %d\n", a);
   printf ("*p: %d\n", *p);
   printf ("**pp: %d\n", **pp);
   printf ("***ppp: %d\n", ***ppp);
}
```

https://onlinegdb.com/rkMJZsJcN

variabilă	adresă	valoare stocată
a	0x7ffc1d19e2e4	23
p	0x7ffc1d19e2e8	0x7ffc1d19e2e4
pp	0x7ffc1d19e2f0	0x7ffc1d19e2e8
ppp	0x7ffc1d19e2f8	0x7ffc1d19e2f0

```
-----Dereferentiere-----
a: 23
*p: 23
**pp: 23
***ppp: 23
```

Dereferențiere multiplă - de ce?

```
main()

main(int argc, char * argv[])

main(int argc, char ** argv)

int main()

int main(void)

int main(int argc, char * argv[])

int main(int argc, char ** argv)

int main(int argc, char * argv[], char *envp[])

int main(int argc, char ** argv, char ** envp)
```

argv[] este o matrice de pointeri. Fiecare membru al argv[] este un pointer către un șir de caractere. La transmiterea matricilor de pointeri către funcții este nevoie de dereferențiere multiplă.

Pointeri ca matrici

```
char * p = & a;
char fb = p[3];
char sb = p[2];
char tb = p[1];
char frb = p[0];
```

O caracteristică a pointerilor este că pot fi apelați cu indice, ca și cum ar fi o matrice, indiferent de definirea unei matrici sau nu. Rezultatul este ceea ce este stocat în următoarea adresă de memorie.

Dacă pointerul ar fi fost de tip int, rezultatul ar fi fost următoarele 4 adrese de memorie, unite pentru a deveni numărul întreg corespunzător.

Variabila noastră avea doar 4 octeți. Pointerul nostru, fiind de tip char va accesa un singur octet. Prin apelarea succesiva a pointerului cu index-ul între 0 și 3 obținem toți octeții în care este stocat numărul a.

Putem solicita conținutul adreselor de memorie dincolo de variabila a, pointerul le returnează fără probleme. Pointerul este de tip char și nu are nici o legătură cu faptul că în primele 4 adrese este stocată o variabilă, el returnează ceea ce îi solicităm. Evident, ceea ce se găsește dincolo de al 4-lea octet nu are sens pentru noi.

Ce se întâmplă dacă mergem mai departe de al 4-lea octet?

octet 1: 10010001
octet 2: 00101110
octet 3: 11011000
octet 4: 01111000
octet 5: 00111100
octet 6: 01001110
octet 7: 00011001
octet 8: 00001101
octet 9: 11111101
octet 10: 01111111
octet 11: 00000000

Pointeri generici și utilizarea lor

Un pointer stochează o adresă de memorie, indiferent de tipul acestuia.

Tipul pointerului este necesar doar pentru operația de derefderențiere. Trebuie să știm câți octeți “citim” din memorie începând cu adresa stocată în pointer.

Dacă NU avem nevoie de dereferențiere, putem renunța la tip. Anumite funcții NU au nevoie de operatia de dereferențiere în interiorul lor, astfel, acestea pot opera doar cu adresele stocate. Acestea se numesc funcții generice pentru că pot primi orice fel de pointer.

Evident, după aplicarea funcției, rezultatul acesteia va trebui utilizat în alte operații și va avea nevoie de dereferențiere. Aceasta se va face prin convertirea pointerului către tipul cerut.

```
void *malloc(size_t size)
```

Funcția malloc alocă o anumită cantitate de memorie (în heap). Malloc are nevoie de un singur parametru, numărul de octeți pe care dorim să ii obținem.

Malloc va returna un pointer către baza zonei de memorie alocată. În această zonă de memorie vom dori să scriem variabile, acestea vor avea un tip. Malloc nu trebuie să știe ce vom scrie în zona respectivă dar trebuie să ne ofere un pointer către baza acelei zone. De aceea, malloc returnează un pointer generic (doar adresa, fără tip), pe care va trebui să îl convertim la tipul de pointer care ne trebuie.

```
newArr=(int*)malloc(size*sizeof(int));
```

size_t (stdlib.h)

size_t este un tip de date definit în stdlib.h care este returnat de operatorul sizeof

Acesta este un tip de date asemănător cu unsigned int, definit în funcție de arhitectura sistemului de operare.

size_t este garantat ca fiind suficient de mare pentru a conține dimensiunea oricărei variabile definite.

size_t nu este niciodată negativ.

size_t este utilizat în multe situații de funcțiile frecvent utilizate în c.

Utilitatea pointerilor

Transferul de informații între funcții

Returnarea mai multor variabile dintr-o funcție

Permit o modalitate alternativă de accesare a elementelor dintr-o matrice sau dintr-un șir

Permit trimiterea matricilor ca argumente ale funcțiilor

Sunt utilizați la construcția tipurilor complexe de date (arbori, liste înlănțuite, grafuri, structuri)

Sunt utilizați la alocarea dinamică a memoriei.

Permit pseudo-polimorfism (pointeri către funcții).

Permit utilizarea funcțiilor de tip call-back

Șiruri de caractere

Șiruri de caractere în C

Un șir de caractere în C este un șir de date de tip char care conține un număr de caractere egal cu numărul de caractere din textul atribuit la care se adaugă un caracter suplimentar numit caracter nul (null character, '\0'). Terminatorul null corespunde cu codul ASCII 0 (nu codul caracterului 0 care este 48).

c	a	r	a	c	a	t	i	t	a	\0
1	2	3	4	5	6	7	8	9	10	11

https://onlinegdb.com/SyWBvkGw_

```
#include <stdio.h>

int main()
{
  char stra[] = {'c', 'a', 'r', 'a', 'c', 'a', 't', 'i', 't', 'a'};
  char strb[] = "caracatita";

  printf ("stra: %ld\n", sizeof(stra)/sizeof(stra[0]));
  printf ("strb: %ld\n", sizeof(strb)/sizeof(strb[0]));
  return 0;
}
```

sir de char fără terminație

terminația este adăugată automat la declarația șirului de caractere în acest mod

siruri1.c

```
a: 10
b: 11
```

Dimensiunea unui șir: strlen

int strlen(const char *str);

Terminatorul șirului nu este printabil dar ocupă o poziție din șir. Astfel, un șir de caractere definit ca având lungimea 50 va putea conține în realitate 49 de caractere “utile”.

https://onlinegdb.com/ds8jPMDo8

```c
#include <stdio.h>
#include <string.h>

int main()
{
  char strb[] = "caracatita";
  int i;

  printf ("sizeof returneaza: %ld\n", sizeof(strb));
  printf ("strlen returneaza: %ld\n", strlen(strb))

  for (i = 0; i < sizeof(strb); i++) {
    printf ("cr: %c numar: %d\n", strb[i], strb[i]);
  }

  return 0;
}
```

siruri2.c

```
sizeof returneaza:
11
strlen returneaza:
10
cr: c - numar: 99
cr: a - numar: 97
cr: r - numar: 114
cr: a - numar: 97
cr: c - numar: 99
cr: a - numar: 97
cr: t - numar: 116
cr: i - numar: 105
cr: t - numar: 116
cr: a - numar: 97
cr:  - numar: 0
```

sizeof este un operator care se aplica pe orice șir/matrice și nu “știe” că este vorba de un șir de caractere; de aceea va returna dimensiunea sa reala, incluzând ultimul caracter (terminatorul nul).

Biblioteca string.h conține funcții speciale de operare cu șiruri de caractere care știu să evite terminatorul null în operații (**strlen** returneaza dimensiunea corectă)

Dimensiunea unui șir: strlen

int strlen(const char *str);

siruri3.c

```
#include <stdio.h>
#include <string.h>

int main()
{

  char stra[] = {'c', 'a', 'r','a','c','a','t','i','t','a'};
  char strb[] = "caracatita";
  char strc[] = {'c', 'a', 'r','a','c','a','t','i','t','a','\0'};

  printf ("stra: %ld\n", sizeof(stra)/sizeof(stra[0]));
  printf ("strb: %ld\n", sizeof(strb)/sizeof(strb[0]));
  printf ("strc: %ld\n", sizeof(strc)/sizeof(strc[0]));
  printf ("strlen stra: %ld\n", strlen(stra));
  printf ("strlen strb: %ld\n", strlen(strb));
  printf ("strlen strc: %ld\n", strlen(strc));

  return 0;
}
```

https://onlinegdb.com/r1U1_yfvO

```
stra: 10
strb: 11
strc: 11
strlen stra: 20
strlen strb: 10
strlen strc: 10
```

strlen stra: 20 arată o dimensiune nepotrivită. În lipsa terminatorului nul, funcțiile care manevrează șiruri de caractere dau rezultate neașteptate și de multe ori erori, în acest caz rezultatul depinde mult de compilator.

Probabil că stra și strb sunt una în continuarea celeilalte în stack iar strlen se oprește la terminatorul nul lui strb.

Atribuirea șirurilor de caractere

Atribuirea șirurilor de caractere se poate face fie la declararea acestora, fie după declarare.

După declarare, atribuirea se poate face doar caracter cu caracter sau prin utilizarea funcțiilor de tipul strcpy. Pentru că un șir de caractere este o matrice și matricile în C sunt ne-atribuibile.

https://onlinegdb.com/SJB7_kzvd

```c
#include <stdio.h>
#include <string.h>

int main()
{

  char stra[] = {'c', 'a', 'r','a','c','a','t','i','t','a'};
  stra = "nautilus";
  printf ("stra este: %s\n", stra);

  return 0;
}
```

siruri4.c

```
string_asignment.c: In function 'main':
string_asignment.c:9:9: error: assignment to expression with array type
    stra = "nautilus";
    ^
```

Șiruri de caractere și pointeri

https://onlinegdb.com/BJVcu1zv_

```c
#include <stdio.h>

int main(void) {

 char *sirPtr = "Cine a pus ulei pe clanta";

 printf("%s\n", sirPtr);

 sirPtr++;
 printf("%s\n", sirPtr);
 sirPtr++;
 printf("%s\n", sirPtr);
 sirPtr++;
 printf("%s\n", sirPtr);
 sirPtr--;
 printf("%s\n", sirPtr);
 sirPtr--;
 printf("%s\n", sirPtr);
 sirPtr--;
 printf("%s\n", sirPtr);

 return 0;
}
```

"%s" nu are nevoie de dereferențiere pentru că se așteaptă la un pointer

pointer_string1.c

```
Cine a pus ulei pe clanta
ine a pus ulei pe clanta
ne a pus ulei pe clanta
e a pus ulei pe clanta
ne a pus ulei pe clanta
ine a pus ulei pe clanta
Cine a pus ulei pe clanta
```

https://onlinegdb.com/ryjkF1GwO

```c
#include <stdio.h>

int main(void) {
 char *sirPtr = "Cine a pus ulei pe clanta";

 printf("%s\n", sirPtr);

 printf("%c\n", *sirPtr);
 sirPtr++;
 printf("%c\n", *sirPtr);
 sirPtr++;
 printf("%c\n", *sirPtr);
 sirPtr++;
 printf("%c\n", *sirPtr);
 sirPtr--;
 printf("%c\n", *sirPtr);
 sirPtr--;
 printf("%c\n", *sirPtr);
 sirPtr--;
 printf("%c\n", *sirPtr);

 return 0;
}
```

"%c" are nevoie de dereferențiere pentru că așteaptă un char, nu un pointer.

pointer_string2.c

```
Cine a pus ulei pe clanta
C
i
n
e
n
i
C
```

Șiruri de caractere și pointeri

pointer_string3.c

```c
#include <stdio.h>

int main(void) {

 char *sirPtr = "Cine a pus ulei pe clanta";

 printf("%s\n", sirPtr);
 printf("%s\n", sirPtr+1);
 printf("%s\n", sirPtr+7);



 printf("%c\n", *sirPtr);
 printf("%c\n", *(sirPtr+1));
 printf("%c\n", *(sirPtr+7));
 return 0;
}
```

https://onlinegdb.com/H1t8K1zvu

```
Cine a pus ulei pe clanta
ine a pus ulei pe clanta
pus ulei pe clanta
C
i
p
```

%s (string), sirPtr (pointer către primul caracter din șir - de fapt sirPtr + 0). Se va imprima până la întâlnirea terminatorului NULL.

%s (string), sirPtr + 1 (pointer către al doiea caracter din șir). Se va imprima până la întâlnirea terminatorului NULL.

%s (string), sirPtr + 7 (pointer către al optulea caracter din șir). Se va imprima până la întâlnirea terminatorului NULL.

%c (caracter), *sirPtr (dereferentierea pointerului către primul caracter din șir). Se va imprima caracterul către care trimite pointerul.

%c (caracter), *(sirPtr+1) (dereferentierea pointerului către cel de-al doilea caracter din șir). Se va imprima caracterul către care trimite pointerul.

%c (caracter), *(sirPtr+1) (dereferentierea pointerului către cel de-al doilea caracter din șir). Se va imprima caracterul către care trimite pointerul.

char *strcpy(char *dest, const char *src)

Copiază șirul de caractere către care trimite pointerul src în șirul de caractere către care trimite pointerul dest.

siruri7.c

```c
#include <stdio.h>
#include <string.h>

int main () {
  char str1[20];
  char str2[20];

  strcpy(str1, "caracatita");
  strcpy(str2, "nautilus");

  printf ("str1 este: %s\n", str1);
  printf ("str2 este: %s\n", str2);

  return 0;
}
```

https://onlinegdb.com/SJKwjyMPO

```
str1 este: caracatita
str2 este: nautilus
```

https://onlinegdb.com/B1uWBhw9V

strcpy este frecvent folosită ca funcție de atribuire pentru șiruri.

La declararea unui șir de caractere:

```c
char str1[20];
```

compilatorul va inițializa un șir de 20 de char și le va umple cu caracterul terminator nul.

```c
strcpy(str1, "caracatita");
```

va copia șirul de caractere “caracatita” în primele 11 (10 + terminatorul null) în str1.

char *strncpy(char *dest, const char *src, size_t n)

siruri9.c

```c
#include <stdio.h>
#include <string.h>

int main()
{

  char stra[] = "caracatita";
  char strb[] = "nautilus";

  printf ("strlen stra este: %ld\n", strlen(stra));
  printf ("sizeof stra este: %ld\n", sizeof(stra));

  strncpy(stra, strb,8);
  printf ("stra este: %s\n", stra);
  printf ("strlen stra este: %ld\n", strlen(stra));
  printf ("sizeof stra este: %ld\n", sizeof(stra));
  printf ("stra[8] este: %c\n", stra[8]);
  printf ("stra[9] este: %c\n", stra[9]);
  return 0;
}
```

https://onlinegdb.com/ryp383vcN

```
strlen stra este: 10
sizeof stra este: 11
stra este: nautilusta
strlen stra este: 8
sizeof stra este: 11
stra[8] este: t
stra[9] este: a
```

strncpy copiază doar numărul de caractere corespunzător parametrului de intrare n. NU adaugă terminator nul. De aceea, ultimele două caractere rămân parte din șir.

probleme în utilizarea strcpy

siruri10.c

```c
#include <stdio.h>
#include <string.h>

int main()
{

  char stra[] = "caracatita";
  char strb[] = "nautilus nu este octopod";
  char strc[] = "ambele sunt cefalopode";

  printf ("strlen stra este: %ld\n", strlen(stra));
  printf ("sizeof stra este: %ld\n", sizeof(stra));

  strcpy(stra, strb);
  printf ("stra este: %s\n", stra);
  printf ("strc este: %s\n", strc);
  printf ("strlen stra este: %ld\n", strlen(stra));
  printf ("sizeof stra este: %ld\n", sizeof(stra));

  return 0;
}
```

https://onlinegdb.com/rk45PnDqN

Utilizarea funcției strcpy trebuie făcută cu mare atenție pentru a nu solicita copierea unui șir mai mare într-un șir mai mic.

Consecința se numește “buffer overrun”

```
strlen stra este: 10
sizeof stra este: 11
stra este: nautilus nu este octopod
strc este:  este octopod
strlen stra este: 24
sizeof stra este: 11
```

Faptul că strb este mai lung decât stra face ca o parte din conținutul său să intre peste următoarea variabilă din stack (strc) și să o suprascrie complet. Comportamentul depinde de compilator si executie.

probleme în utilizarea strncpy

siruri11.c

```c
#include <stdio.h>
#include <string.h>

int main()
{

  char stra[] = "caracatita";
  char strb[] = "nautilus nu este octopod";
  char strc[] = "ambele sunt cefalopode";

  printf ("strlen stra este: %ld\n", strlen(stra));
  printf ("sizeof stra este: %ld\n", sizeof(stra));

  strncpy(stra, strb, 16);
  printf ("stra este: %s\n", stra);
  printf ("strc este: %s\n", strc);
  printf ("strlen stra este: %ld\n", strlen(stra));
  printf ("sizeof stra este: %ld\n", sizeof(stra));

  return 0;
}
```

https://onlinegdb.com/SktN_hP9E

strncpy nu adaugă de la sine un terminator nul.

Utilizarea neatentă conduce din nou la buffer overrun.

```
strlen stra este: 10
sizeof stra este: 11
stra este: nautilus nu estee sunt
cefalopode
strc este:  estee sunt cefalopode
strlen stra este: 33
sizeof stra este: 11
```

16 caractere este mai puțin decât lungimea lui strb (24) dar mai mult decât lungimea lui stra (10). Consecință: nu mai există nici un terminator nul. Primul disponibil este abia la finalul șirului de caractere strc care se găsește în stack după strb. Ca atare, citirea șirului de caractere stra dă rezultate false (un fel de concatenare între strb și ceea ce a rămas din strc).

Citirea șirurilor de la consolă

int scanf(const char *format, ...)

este o funcție generală care permite citirea de la consolă a unor date și atribuirea acestora unei variabile.

scanf1.c

```
#include <stdio.h>

int main()
{
   int s;
   float b;
   char c;
   printf("Introduceti un numar intreg:");
   scanf ("%d", &s);
   printf ("Ati introdus numarul %d\n",s);
   printf("Introduceti un numar rational:");
   scanf ("%f", &b);
   printf ("Ati introdus numarul %f\n",b);
   printf("Introduceti un caracter:");
   scanf (" %c", &c);
   printf ("Ati introdus caracterul %c\n",c);
   return 0;
}
```

https://onlinegdb.com/ry_y_P92X

```
Introduceti un numar intreg:12
Ati introdus numarul 12
Introduceti un numar rational:32.45
Ati introdus numarul 32.450001
Introduceti un caracter:d
Ati introdus caracterul d
```

Citirea șirurilor de la consolă

int scanf(const char *format, ...)

Până acum am utilizat scanf pentru citirea numerelor. Variabilele în care așteptam să se regăsească valorile citite trebuiau transmise către scanf ca referințe pentru a fi disponibile și după încheierea execuției funcției.

https://onlinegdb.com/rk8WU6xwO

scanf2.c

```
#include <stdio.h>
#include <string.h>

int main()
{
  char str[100];
  printf("introduceti o expresie:");
  scanf("%s", str);
  printf("introdus: %s",str);
  return 0;
}
```

```
introduceti o expresie:cine a pus
introdus: cine
```

Șirurile de caractere însă sunt în realitate pointeri constanți. Astfel, când încercăm să obținem un șir de caractere, nu transmitem o referință către variabila care va conține șirul ci direct variabila (ea este deja referință).

scanf3.c

```
int main()
{
  char str[100];
  printf("introduceti o expresie:");
   scanf("%[^\n]s", cuvant);
  printf("introdus: %s",str);
  return 0;
}
```

```
introduceti o expresie:cine a pus
introdus: cine a pus
```

Declarată ca în codul de mai sus, scanf va citi până va întâlni primul caracter cu valoare de delimitator (spațiu, tab). Dacă dorim să introducem un șir de caractere care conține spații, specificatorul %s va trebui modificat astfel:

```
scanf("%[^\n]s", cuvant);
```

Aceasta va determina scanf să se oprească doar la primirea caracterului return (\n).

buffer overrun cu scanf

Orice funcție care conduce la copierea unui șir de caractere într-o anumită variabilă este susceptibilă de buffer overrun.

https://onlinegdb.com/HJ-0hJfvO

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char cuvant[2];

    printf("cuvant:");
    scanf("%[^\n]s", cuvant);
    printf("%s\n",cuvant);
    return 0;
}
```

scanf4.c

https://onlinegdb.com/H1GI61Mv_

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char cuvant[2];
    char str[10];
    printf("cuvant:");
    scanf("%[^\n]s", cuvant);
    printf("%s\n",cuvant);
    return 0;
}
```

scanf5.c

```
introduceti un cuvânt:mare
mare
*** stack smashing detected
***: <unknown> terminated
Aborted (core dumped)
```

```
cuvânt:mare
mare
```

```
cuvânt:mare de tot
mare de tot
*** stack smashing detected ***:
terminated
Aborted (core dumped)
```

Simpla definire a unei variabile suplimentare imediat după cuvant[2] a determinat programul sa nu mai dea eroare. Atentie! în continuare programul este în starea de buffer overrun: cuvant[] trebuie sa nu aibă decât două caractere iar la consolă au fost introduse 4. Și cel de-al doilea program poate fi forțat să depășească spațiul din stack dacă se introduce un șir de caracter mai mare.

*** stack smashing detected ***

Stack smashing reprezintă scrierea datelor dincolo de spațiul din stack rezervat de compilator pentru variabilele unei anumite funcții. În primul caz, singura variabilă definită era cuvant[2] iar apoi spațiul dedicat variabilelor funcției main() se termina. Introducerea unui șir mai mare de 2 a determinat încercarea de a suprascrie conținutul memoriei stack care nu aparținea funcției main(). Compilatorul a introdus o metodă de protecție împotriva acestor încercări și programul a fost oprit.

în cel de-al doilea caz, declararea celei de-a doua variabile (str[10]) a mărit spațiul alocat în stack pentru funcția main() cu 10 octeti, astfel încât, deși a avut loc suprascrierea memoriei rezervate variabilei atr (buffer overrun), totusi, nu a fost depășit spațiul rezervat funcției. Dacă introducem un șir de caractere mai lung decât totalitatea spațiului ocupat de variabilele definite în main() (să zicem un șir de 20 de caractere), programul se va opri din nou cu același mesaj.

Protecția poate fi dezactivată (în cazul gcc cu utilizarea opțiunii de compilare -fno-stack-protector). Dacă este dezactivată, programul poate rula corect, sau, la valori foarte mari ale șirului, se va opri cu eroarea segmentation fault care este sesizată direct de nucleul sistemului de operare.

```
gcc -Wall -fno-stack-protector reading_strings3.c -o reading_strings3
```

```
cuvânt:mare
mare
```

```
cuvânt:celmaimarecuvantdinlume
celmaimarecuvantdinlume
Segmentation fault (core dumped)
```

puts și gets

puts și gets sunt funcții speciale pentru citirea, respectiv scriere șirurilor de caractere.

https://onlinegdb.com/ByWhakGwu

```c
#include <stdio.h>

int main()
{

   char cuvant[100];
   puts("introduceti un cuvânt:");
   gets(cuvant);
   puts(cuvant);

   return 0;
}
```

puts_gets.c

Compilarea acestui program produce o serie de warning-uri.

Funcția gets a fost eliminata din standardul C în anul 1999 din cauza pericolului de a introduce vulnerabilități majore în programe prin erori de tip buffer overrun sau stack overflow.

```
reading_strings4.c: In function 'main':
reading_strings4.c:9:5: warning: implicit declaration of function 'gets';
did you mean 'fgets'? [-Wimplicit-function-declaration]
     gets(cuvant);
     ^~~~
     fgets
/usr/bin/ld: /tmp/ccU7ge6a.o: in function `main':
reading_strings4.c:(.text+0x30): warning: the `gets' function is dangerous
and should not be used.
```

Citirea șirurilor de la consolă - fgets

https://onlinegdb.com/BJRIRJfDu

```c
#include <stdio.h>
#define MAX 15

int main()
{
    char astr[MAX];
    puts("introduceti un sir:");
    fgets(astr, MAX, stdin);
    printf("string is: %s\n", astr);

    return 0;
}
```

puts_fgets.c

```
introduceti un sir:
vulpea rapida
string is: vulpea rapida
```

```
introduceti un sir:
vulpea rapida sare peste cainele lenes
string is: vulpea rapida
```

char *fgets(char *str, int n, FILE *stream)

str : Pointer către un șir de caractere unde se va copia ceea ce se citește
n : Numărul maxim de caractere care va fi copiat în str, inclusiv terminatorul nul.
***stream :** Pointer către un fișier deschis (pentru citirea de la consolă se folosește stdin - standard input)

funcția returnează pointerul str

Datorită trimiterii numărului de caractere ca parametru de intrare, fgets este o funcție mai sigură decât gets sau scanf. Cunoscând lungimea dorită, fgets poate limita corect numărul de caractere, ocolind depășirea spațiului disponibil.

https://en.wikipedia.org/wiki/The_quick_brown_fox_jumps_over_the_lazy_dog

pointeri către șiruri

https://onlinegdb.com/H1fT1xMP_

În cazul oricărui șir/matrice, variabila definită este un pointer către primul element al acesteia. Astfel, la definirea unui șir

char str[];

str reprezintă un array de char - uri care se va transforma in pointer la trimiterea către o funcție.

Șirurile pot fi definite ca pointeri direct.

```
#include <stdio.h>
#include <string.h>

int main()
{
  char *strp = "caracatita";
  printf ("strp este: %s\n", strp);
  return 0;
}
```

scanf_init.c

https://onlinegdb.com/HJe5Jefv_

```
#include <stdio.h>
#include <string.h>

int main()
{
  char *strp;
  scanf("%s", strp);
  printf ("strp este: %s\n", strp);
  return 0;
}
```

scanf_null.c

```
caracatita
strp este: (null)
```

Un pointer astfel definit NU poate fi folosit în funcția scanf pentru că nu este inițializat. Adica, este alocată memoria pentru a stoca o adresă a altei variabile dar încă nu este nimic stocat în acea memorie.

```
char arry[100];
char *strp = arry;
scanf("%s", strp);
```

Compararea șirurilor de caractere

int strcmp(const char *str1, const char *str2)

strcmp1.c

```c
#include <stdio.h>
#include <string.h>

int main () {
  char str1[20];
  char str2[20];
  int result;

  strcpy(str1, "caracatita");
  strcpy(str2, "CARACATITA");

  result = strcmp(str1, str2);

  if(result > 0) {
     printf("str1 > str2 (result: %d)\n", result);
  } else if(result < 0) {
     printf("str1 < str2 (result: %d)\n", result);
  } else {
     printf("str1 == str2 (result: %d)\n", result);
  }
  return 0;
}
```

https://onlinegdb.com/r1aVHpD5N

strcmp returnează prima diferență nenulă dintre două caractere aflate pe aceeași poziție în cele două șiruri de caractere

65	41	101 A	A
66	42	102 B	B
67	43	103 C	C
68	44	104 D	D
69	45	105 E	E
70	46	106 F	F
71	47	107 G	G

97	61	141 a	a
98	62	142 b	b
99	63	143 c	c
100	64	144 d	d
101	65	145 e	e
102	66	146 f	f
103	67	147 g	g

99 - 67 = 32

```
str1 > str2 (result: 32)
```

`char * strcat(char *dest, const char *src)`

strcat e o funcție care concatenează cel de-al doilea șir la capătul primului.

strcat1.c

```c
#include <stdio.h>
#include <string.h>

int main() {
  char str1[100] = "If it bleeds, ";
  char str2[] = "we can kill it.";

  strcat(str1, str2);

  printf ("%s\n", str1);

  return 0;
}
```

https://onlinegdb.com/rJBhTmyPO

If it bleeds, we can kill it.

Funcția strcat poate fi apelată fără a se solicita returnarea într-o anumită variabilă.

Dacă este apelată cu returnare într-o variabilă (care trebuie să fie de tip pointer către char) acea variabilă va primi un pointer către primul argument al funcției.

strcat2.c

```c
#include <stdio.h>
#include <string.h>

int main() {
  char str1[100] = "If it bleeds, ";
  char str2[] = "we can kill it.";

  char * str3;
  str3 = strcat(str1, str2);

  printf ("%s\n", str3);

  printf ("%p\n", str1);
  printf ("%p\n", str3);

  return 0;
}
```

https://onlinegdb.com/HyyrTX1Du

If it bleeds, we can kill it.
0x7ffd78a155f0
0x7ffd78a155f0

Implementare strcat

mystrcat.c

```c
#include <stdio.h>

char *mystrcat(char *dest, const char *src)
{
   int index_dest = 0, dest_len = 0, src_len = 0, index_src=0;

   while (dest[index_dest++]) {
       dest_len++;
   }

   while (src[index_src++]) {
       src_len++;
   }

   for (index_dest = 0; index_dest < src_len; index_dest++) {
       dest[dest_len] = src[index_dest];
       dest_len++;
   }

   return dest;
}

int main() {
 char str1[100] = "If it bleeds, ";
 char str2[] = "we can kill it.";

 mystrcat(str1, str2);
 printf ("%s\n", str1);

 return 0;
}
```

https://onlinegdb.com/By9_rgGvd

Numărăm câte caractere are șirul dest

Numărăm câte caractere are șirul src

Copiem caracterele din șirul src (de la 0 până la src_len) în șirul destinație (de la dest_len până la dest_len + src_len)

```
If it bleeds, we can kill it.
```

strtof, strtod, strtold

strtof.c

```c
#include <stdio.h>
#include <stdlib.h>

int main () {
  char str[30] = "20.30300 si ceva litere";
  char *ptr;
  double ret;

  ret = strtof(str, &ptr);
  printf("Numarul flotant este %f\n", ret);
  printf("Restul sirului:  %s", ptr);

  return(0);
}
```

https://onlinegdb.com/S19DmZzwO

```
Numarul flotant este 20.302999
Restul sirului:   si ceva litere
```

șirul care va fi analizat

pointer cu ajutorul căruia se poate obține restul șirului

variabila în care se va returna numărul flotant.

Cele două funcții au același comportament, ele diferă doar prin tipul de dată flotantă pe care îl returnează.

strtof și strtod sunt capabile să “extragă” numărul flotant aflat la începutul unui șir de caractere, returnând în același timp și restul șirului.

atoi, atof, atol

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
   //atoi (ASCII to Integer)
   const char* intStr = "123";
   int integerValue = atoi(intStr);
   printf("atoi: %d\n", integerValue);

   //atof (ASCII to Float)
   const char* floatStr = "123.45";
   float floatValue = atof(floatStr);
   printf("atof: %f\n", floatValue);

   //atol (ASCII to Long)
   const char* longStr = "1234567890";
   long longValue = atol(longStr);
   printf("atol: %ld\n", longValue);
   return 0;
}
```

ato… sunt funcții care nu verifică succesul conversiei, dacă nu recunosc valori numerice vor returna 0

https://onlinegdb.com/dsjiCasMK

```
atoi: 123
atof: 123.449997
atol: 1234567890
```

snprintf

snprintf funcționează ca printf cu deosebirea că destinația acesteia este o variabilă de tip șir de caractere, nu consola.

```
#include <stdio.h>

int main() {
   float floatValue = 123.45;
   char buffer[20];

   int charsWritten = snprintf(buffer, sizeof(buffer), "%f", floatValue);

   if (charsWritten >= 0 && charsWritten < sizeof(buffer)) {
       // Conversia a fost realizată cu succes
       printf("Valoarea convertită: %s\n", buffer);
   } else {
       // Eroare de conversie
       printf("Eroare la conversie.\n");
   }

   return 0;
}
```

https://onlinegdb.com/0lzGl7ac2

Valoarea convertită: 123.449997

Fișiere

Scriere și citire din fișiere

Citire din fișiere	Scriere in fisiere
fread() size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream) Citește un array de count elemente, fiecare cu dimensiunea size (în octeți) și le stochează într-un bloc de memorie specificat de pointerul ptr.	**fwrite()** size_t fwrite (const void * ptr, size_t size, size_t count, FILE * stream); Scrie un array de count elemente, fiecare cu dimensiunea size în octeți, din blocul de memorie specificat de pointerul ptr în poziția curentă a fluxului.
fscanf() int fscanf (FILE * stream, const char * format, ...); Citește date din flux conform parametrului format și le stochează în pointeri predefiniți și specificați în argumentele funcției	**fprintf()** int fprintf (FILE * stream, const char * format, ...); Scrie în flux șirul comnstruit pe baza argumentului “format” și a variabilelor transmise. Similar cu printf
fgetc() int fgetc (FILE * stream); Returnează caracterul (octetul) care se află la poziția curentă a fluxului. Apoi, pointerul intern al fluxului este mutat la următoarea poziție	**fputc()** int fputc (int character, FILE * stream); Scrie un *octet (character)* în *stream* și mută poziția pointerului cu un octet.
fgets() char * fgets (char * str, int num, FILE * stream); Citește un număr arbitrar de caractere din stream până se ajunge la num-1 octeți, un caracter de tip newline sau sfârșitul fișierului.	**fputs()** int fputs (const char * str, FILE * stream); Scrie șirul către care trimite pointerul str în stream, începând de la poziția curentă.

structura opacă _IO_FILE linux (pentru fiecare sistem de operare este diferită)

```
struct _IO_FILE
{
 int  flags;/* High-order word is  IO MAGIC; rest is flags. */
 char * IO read ptr;/* Current read pointer */
 char * IO read end;/* End of get area. */
 char * IO read base;/* Start of putback+get area. */
 char * IO write base;/* Start of put area. */
 char * IO write ptr;/* Current put pointer. */
 char * IO write end;/* End of put area. */
 char * IO buf base;/* Start of reserve area. */
 char *_IO_buf_end;/* End of reserve area. */

 /* The following fields are used to support backing up and undo. */
 char * IO save base;
 char * IO backup base;
 char *_IO_save_end;

 struct  IO marker * markers;
 struct  IO FILE *_chain;
 int  fileno;
 int  flags2;
 unsigned short  cur column;
 signed char  vtable_offset;
 char  shortbuf[1];
  IO_lock_t *_lock;
};
```

Pointeri către stream buffer

Lista înlănțuită cu legături cu streamurile standard

File handle, returnat de sistemul de operare

```
FILE *fopen(const char *filename, const char *mode)
```

Opțiuni de deschidere a fișierului în mod text (ASCII)

Mod		Descriere	Dacă fișierul există	Dacă fișierul nu există
r	citire	Deschide pentru citire	Citește de la început	fopen() returnează NULL
w	scriere	Deschide pentru scriere	Suprascrie conținutul	Fișierul va fi creat gol
a	adăugare	Deschide pentru adăugare, datele sunt adăugate la finalul fișierului	Adaugă la conținut	Fișierul va fi creat gol
r+	citire extinsă	Deschide pentru citire și scriere	Citește de la început	Eroare
w+	scriere extinsă	Crează pentru citire și scriere	Suprascrie conținutul	Fișierul va fi creat gol
a+	adăugare extinsă	Deschide pentru citire și adăugare	Adaugă la finalul fișierului	Fișierul va fi creat gol

```
fp = fopen ("file.txt", "w+");
```

```
FILE *fopen(const char *filename, const char *mode)
```

Opțiuni de deschidere a fișierului în mod binar

Mod		Descriere	Dacă fișierul există	Dacă fișierul nu există
rb	citire	Deschide pentru citire	Citește de la început	fopen() returnează NULL
wb	scriere	Deschide pentru scriere	Suprascrie conținutul	Fișierul va fi creat gol
ab	adăugare	Deschide pentru adăugare, datele sunt adăugate la finalul fișierului	Adaugă la conținut	Fișierul va fi creat gol
rb+	citire extinsă	Deschide pentru citire și scriere	Citește de la început	Eroare
wb+	scriere extinsă	Crează pentru citire și scriere	Suprascrie conținutul	Fișierul va fi creat gol
ab+	adăugare extinsă	Deschide pentru citire și adăugare	Adaugă la finalul fișierului	Fișierul va fi creat gol

Stocarea într-un fișier text a unui număr întreg (ex: 1234) se face cu ajutorul a 4 caractere ASCII, 1, 2, 3, 4 (adica 00000001, 00000010, 00000011, 00000100 - 4 octeți). Dar, în același timp, reprezentat ca număr întreg, 1234 este 0000010011010010 care poate fi stocat pe doar 2 octeți,

Numerele sunt stocate mult mai eficient în format binar direct. Pentru a le putea trimite în fișier și a le putea citi din fișier este nevoie ca pointerii să se mute corespunzător, conform variabilelor citite. Aceasta se poate face dacă fișierele se deschid în mod binar.

Nimic nu ne împiedică să citim, octet cu octet, un fișier ASCII ca fișier binar, inspectând fiecare pattern citit și interpretându-l ca atare.

Scrierea în fișier (putc)

```
int putc ( int character, FILE * stream );
```

file_write_pointers.c

```c
#include <stdio.h>

int main()
{
 FILE *fp;
 int c = 'a';
 fp = fopen("test.txt", "w");
 if (fp) {
     printf("read_ptr: %p\n", fp->_IO_read_ptr);
     printf("read_ptr: %p\n", fp->_IO_write_ptr);
     c = putc(c,fp);
     printf("Char: %c\n", c);
     printf("read_ptr addr: %p\n", fp->_IO_read_ptr);
     printf("write_ptr: %p\n", fp->_IO_write_ptr);
     printf("read_ptr: %c\n", *(fp->_IO_read_ptr));
     c = putc(c,fp);
     printf("Char: %c\n", c);
     printf("read_ptr addr: %p\n", fp->_IO_read_ptr);
     printf("write_ptr: %p\n", fp->_IO_write_ptr);
     printf("read_ptr: %c\n", *(fp->_IO_read_ptr));
     c = putc(c,fp);
     printf("Char: %c\n", c);
     printf("read_ptr addr: %p\n", fp->_IO_read_ptr);
     printf("write_ptr: %p\n", fp->_IO_write_ptr);
     printf("read_ptr: %c\n", *(fp->_IO_read_ptr));

   }
 fclose(fp);
 return 0;
}
```

https://onlinegdb.com/B1JWyfzD_

Cât timp fișierul este deschis, se mențin informații cu privire la poziția curentă în fișier.

pointerul write_ptr se mută odată cu operația de citire
pointerul read_ptr rămâne pe loc

```
read_ptr: (nil)
read_ptr: (nil)
Char: a
read_ptr addr: 0x7fa5e3450000
write_ptr: 0x7fa5e3450001
read_ptr: a
Char: a
read_ptr addr: 0x7fa5e3450000
write_ptr: 0x7fa5e3450002
read_ptr: a
Char: a
read_ptr addr: 0x7fa5e3450000
write_ptr: 0x7fa5e3450003
read_ptr: a
```

Fișierul conține: aaa

Scrierea în fișier (fputs)

int fputs(const char *str, FILE *stream)

```
#include <stdio.h>

main()
{
  FILE *fp;

  fp = fopen("test.txt", "w");
  fputs("Testing...", fp);
  fclose(fp);
}
```

https://onlinegdb.com/ryxfbfzwu

La mai multe rulări succesive, fișierul test.txt va fi suprascris. Fișierul va conține de fiecare dată doar un șir: Testing...

```
#include <stdio.h>

main()
{
  FILE *fp;

  fp = fopen("test.txt", "a");
  fputs("Testing...", fp);
  fclose(fp);
}
```

https://onlinegdb.com/rkgXt-wyvu

La mai multe rulări succesive, fișierul test.txt va fi deschis iar șirul “Testing…” va fi adăugat.

Pentru fiecare rulare, în fișier se va adăuga un alt rând conținând șirul “Testing…”

Scrierea în fișier (fprintf)

int fprintf (FILE * stream, const char * format, ...);

```
#include <stdio.h>

main()
{
  FILE *fp;

  fp = fopen("test.txt", "w");
  fprintf(fp, "Testing...\n");
  fclose(fp);
}
```

https://onlinegdb.com/B1RLbv1w_

La mai multe rulări succesive, fișierul test.txt va fi suprascris. Fișierul va conține de fiecare dată doar un șir: Testing...

```
#include <stdio.h>

main()
{
  FILE *fp;

  fp = fopen("test.txt", "a");
  fprintf(fp, "Testing...\n");
  fclose(fp);
}
```

https://onlinegdb.com/rkgXt-wyvu

La mai multe rulări succesive, fișierul test.txt va fi deschis iar șirul “Testing…” va fi adăugat.

Pentru fiecare rulare, în fișier se va adăuga un alt rând conținând șirul “Testing…”

Citirea din fișier (getc)

int getc(FILE *stream)

file_read_pointers.c

```c
#include <stdio.h>

int main()
{
 FILE *fp;
 int c;
 fp = fopen("test.txt", "r");

 if (fp) {
     printf("read_ptr: %p\n", fp->_IO_read_ptr);
     printf("read_ptr: %p\n", fp->_IO_write_ptr);
     c = getc(fp);
     printf("Char: %c\n", c);
     printf("read_ptr addr: %p\n", fp->_IO_read_ptr);
     printf("write_ptr: %p\n", fp->_IO_write_ptr);
     printf("read_ptr: %c\n", *(fp->_IO_read_ptr));
     c = getc(fp);
     printf("Char: %c\n", c);
     printf("read_ptr addr: %p\n", fp->_IO_read_ptr);
     printf("write_ptr: %p\n", fp->_IO_write_ptr);
     printf("read_ptr: %c\n", *(fp->_IO_read_ptr));
     c = getc(fp);
     printf("Char: %c\n", c);
     printf("read_ptr addr: %p\n", fp->_IO_read_ptr);
     printf("write_ptr: %p\n", fp->_IO_write_ptr);
     printf("read_ptr: %c\n", *(fp->_IO_read_ptr));

}
 fclose(fp);
 return 0;
}
```

https://onlinegdb.com/SyFnNMGPO

Cât timp fișierul este deschis, se mențin informații cu privire la poziția curentă în fișier.

pointerul read_ptr se mută odată cu operația de citire
pointerul write_ptr rămâne pe loc

```
read_ptr: (nil)
read_ptr: (nil)
Char: a
read_ptr addr: 0x7f93a659f001
write_ptr: 0x7f93a659f000
read_ptr: b
Char: b
read_ptr addr: 0x7f93a659f002
write_ptr: 0x7f93a659f000
read_ptr: c
Char: c
read_ptr addr: 0x7f93a659f003
write_ptr: 0x7f93a659f000
read_ptr:
```

Fișierul conține: abc

Citirea dintr-un fișier (fgets)

```
char * fgets ( char * str, int num, FILE * stream );
```

file_fgets.c

```
#include <stdio.h>
#include <string.h>

int main() {
  char buf[1024];
  FILE *fp;
  size_t nread;

  fp = fopen("testlg.txt", "r");
  if (fp) {
      while (fgets(buf, sizeof(buf), fp)) {
       printf("%s", buf);
       }
      fclose(fp);
  }
  return 0;
}
```

https://onlinegdb.com/HJVLXzGwO

```
citit: Cine bate seara
citit: La fereastra mea
```

fgets se oprește la întâlnirea unui caracter de tip EOF sau la sfârșit de rând.

Deschiderea fișierului

Se citeste din fișier o cantitate de octeți egală cu dimensiunea bufferului

Atâta timp cât fgets returnează altceva decât un pointer NULL, se trimite buf la printf.

Citirea unui fișier (fread)

```
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream)
```

file_ascii_read.c

```
#include <stdio.h>
#include <string.h>

int main() {
   char buf[1024];
   FILE *fp;
   size_t nread;

   fp = fopen("test.txt", "r");
   if (fp) {
       while ((nread = fread(buf, 1, sizeof buf, fp)) > 0) {
           printf("read: %ld:  %s\n",nread, buf);
           memset(buf, '\0', sizeof buf);
       }
       fclose(fp);
   }
   return 0;
}
```

https://onlinegdb.com/rJvqRP1vd

Deschiderea fișierului

Se citeste din fișier o cantitate de octeți egală cu dimensiunea bufferului

Se golește bufferul pentru a se evita cazul în care ceea ce se citește în buffer este mai mic decât ceea ce a fost citit anterior.

Dimensiunea bufferului determină numărul citirilor

read: 43: Primul rand Al doilea rand Al treilea rand	`char buf[1024]`
read: 24: Primul rand Al doilea ra read: 19: nd Al treilea rand	`char buf[24]`

Citirea unui fișier cu dimensiune necunoscută

```
#include <stdio.h>
#include <stdlib.h>

int main()
{

    FILE *fp;
    fp = fopen("bigf.txt", "r");

    fseek(fp, 0, SEEK_END);

    long fsize = ftell(fp);

    fseek(fp, 0, SEEK_SET);   /* same as rewind(f); */

    printf("File size: %ld", fsize);

    char *string = malloc(fsize + 1);
    fread(string, 1, fsize, fp);
    fclose(fp);

    string[fsize] = 0;

    printf ("%s", string);

    return 0;
}
```

Deschidem fișierul

Mută pointerul `read_ptr` acolo unde este finalul fișierului (SEEK_END)

ftell returnează poziția curentă a pointerului. Fiind la final, vom obține numărul de octeți.

Mutăm pointerul la început

Alocăm cantitatea necesară de memorie

Citim întreg conținutul fișierului în string

Adăugăm terminatorul NULL

Structuri

Obiectele reale sunt caracterizate de mai multe proprietăți, de cele mai multe ori exprimate prin tipuri de date diferite. Avem nevoie de o modalitate de a defini tipuri de date complexe care să ne permită descrierea acestora

Nume	Rubin
Nume științific	Corindon
Formulă chimică	Al2O3
Duritate	9
Indice de refracție	1.76
Fotografie	ruby.jpg
Culoare	rosu

Nume	Safir
Nume științific	Corindon
Formulă chimică	Al2O3
Duritate	9
Indice de refracție	1.76
Fotografie	safir.jpg
Culoare	albastru

Nume	Smarald
Nume științific	Beril
Formulă chimică	Be3Al2Si6O18
Duritate	8
Indice de refracție	1.59
Fotografie	smarald.jpg
Culoare	verde

Nume	Diamant
Nume științific	Diamant
Formulă chimică	C
Duritate	10
Indice de refracție	2,41
Fotografie	diamant.jpg
Culoare	incolor

gems_array.c

```c
#include <stdio.h>
#include <string.h>

int main()
{
   int indc;
   char *nume[4];
   char *snume[4];
   char *formula[4];
   int duritate[4];
   float refr[4];
   char *foto[4];
   char *clr[4];

   nume[0] = "Rubin"; nume[1] = "Safir"; nume[2] = "Smarald"; nume[3] = "Diamant";
   snume[0] = "Corindon"; snume[1] = "Corindon"; snume[2] = "Beril"; snume[3] = "Diamant";
   formula[0] = "Al2O3";formula[1] = "Al2O3";formula[2] = "Be3Al2Si6O18"; formula[3] = "C";
   duritate[0] = 9;duritate[1] = 9;duritate[2] = 8;duritate[3] = 10;
   refr[0] = 1.76;refr[1] = 1.76;refr[2] = 1.59;refr[3] = 2.41;
   foto[0] = "ruby.jpg"; foto[1] = "safir.jpg";foto[2] = "smarald.jpg";foto[3] = "diamant.jpg";
   clr[0] = "rosu";clr[1] = "albastru";clr[2] = "verde"; clr[3] = "incolor";

   puts ("Introduceti indicele (intre 0 si 3)\n");
   scanf("%d", &indc);
   printf("Intrarea cu numarul %d\n\n", indc);
   printf("Nume: %s\n",nume[indc]);
   printf("Nume stiintific: %s\n",snume[indc]);
   printf("Formula chimica: %s\n",formula[indc]);
   printf("Duritate: %d\n",duritate[indc]);
   printf("Indice de refr: %f\n",refr[indc]);
   printf("Fotografie: %s\n",foto[indc]);
   printf("Culoare: %s\n",clr[indc]);

   return 0;
}
```

Declarăm câte o matrice unidimensională pentru fiecare dintre proprietățile obiectelor. Fiecare matrice este declarată cu tipul necesar (char[], int, float) și dimensionată astfel încât să ne asigurăm că indicii corespund între ei.

https://onlinegdb.com/S16tBav54

Introduceti indicele (intre 0 si 3)
2
Intrarea cu numarul 2

Nume: Smarald
Nume stiintific: Beril
Formula chimica: Be3Al2(SiO3)6
Duritate: 8
Indice de refr: 1.590000
Fotografie: smarald.jpg
Culoare: verde

Structurile permit gruparea mai multor tipuri de date

struct_stone.c

```c
#include <stdio.h>
#include <string.h>

struct Stone
    {
        char nume[50];
        char snume[50];
        char formula[50];
        int duritate;
        float refr;
        char foto[30];
        char clr[30];
    };


int main()

{
   struct Stone rubin, safir, smarald, diamant;

   strcpy(rubin.nume, "Rubin");
   strcpy(rubin.snume, "Corindon");
   strcpy(rubin.formula, "Al2O3");

   printf ("Nume: %s\n", rubin.nume);
   printf ("Nume stiintific: %s\n", rubin.snume);
   printf ("Formula chimica: %s\n", rubin.formula);

}
```

Structurile se definesc de obicei în afara funcției main() pentru a avea un scop global.

Structurile sunt tipuri complexe de date, nu grupări de variabile. După definirea lor, trebuiesc declarate variabile cu noul tip.

Accesarea membrilor unei structuri (citire și scriere) se face cu ajutorul operatorului punct (.)

```
Nume: Rubin
Nume stiintific: Corindon
Formula chimica: Al2O3
```

https://onlinegdb.com/B1m2rpvc4

Declararea directă a structurilor

struct_stone_direct.c

```c
#include <stdio.h>
#include <string.h>

struct Stone
   {
       char nume[50];
       char snume[50];
       char formula[50];
       int duritate;
       float refr;
       char foto[30];
       char clr[30];
   };

int main()

{
   struct Stone rubin = {"Rubin", "Corindon", "Al2O3", 9, 1.76, "ruby.jpg", "rosu"};
   struct Stone safir = {"Safir", "Corindon", "Al2O3", 9, 1.76, "safir.jpg", "albastru"};
   struct Stone smarald = {"Smarald", "Beril", "Be3Al2Si6O18", 8, 1.59, "smarald.jpg", "verde"};
   struct Stone diamant = {"Diamant", "Diamant", "C", 10, 2.41, "diamant.jpg", "incolor"};

   puts ("Rubin\n");
   printf ("Nume: %s\n", rubin.nume);
   printf ("Nume stiintific: %s\n", rubin.snume);
   printf ("Formula chimica: %s\n", rubin.formula);
   puts ("\nSmarald\n");
   printf ("Nume: %s\n", smarald.nume);
   printf ("Nume stiintific: %s\n", smarald.snume);
   printf ("Formula chimica: %s\n", smarald.formula);
}
```

```
Rubin

Nume: Rubin
Nume stiintific: Corindon
Formula chimica: Al2O3

Smarald

Nume: Smarald
Nume stiintific: Beril
Formula chimica: Be3Al2Si6O18
```

https://onlinegdb.com/BJKQH5WOu

Șirurile din structuri se pot defini și ca pointeri

struct_stone_pointers.c

```
#include <stdio.h>
#include <string.h>

struct Stone
    {
        char *nume;
        char *snume;
        char *formula;
        int duritate;
        float refr;
        char *foto;
        char *clr;
    };


int main()

{
    struct Stone rubin, safir, smarald, diamant;

    rubin.nume =  "Rubin";
    rubin.snume = "Corindon";
    rubin.formula = "Al2O3";

    printf ("Nume: %s\n", rubin.nume);
    printf ("Nume stiintific: %s\n", rubin.snume);
    printf ("Formula chimica: %s\n", rubin.formula);
}
```

https://onlinegdb.com/SJwUDTv5E

Structurile se definesc de obicei în afara funcției main() pentru a avea un scop global.

Structurile sunt tipuri complexe de date, nu grupări de variabile. După definirea lor, trebuiesc declarate variabile cu noul tip.

Accesarea membrilor unei structuri (citire și scriere) se face cu ajutorul operatorului punct (.)

Functionează fără malloc pentru că sirurile sunt declarate în momentul compilării.

```
Nume: Rubin
Nume stiintific: Corindon
Formula chimica: Al2O3
```

Structurile pot fi accesate și cu ajutorul pointerilor

pointers_to_struct.c

```c
#include <stdio.h>
#include <string.h>

struct Stone
   {
       char *nume;
       char *snume;
       char *formula;
       int duritate;
       float refr;
       char *foto;
       char *clr;
   };

int main()

{
   struct Stone *prubin, *psafir, *psmarald, *pdiamant, rubin, safir, smarald, diamant;

   prubin = &rubin; psafir = &safir; pdiamant = &diamant; psmarald = &smarald;

   prubin->nume =  "Rubin";
   prubin->snume = "Corindon";
   prubin->formula = "Al2O3";

   printf ("Nume: %s\n", prubin->nume);
   printf ("Nume stiintific: %s\n", prubin->snume);
   printf ("Formula chimica: %s\n", prubin->formula);
}
```

pointerii se definesc cu același tip de date (structura definită mai sus)

Pointerilor li se atribuie adresa fiecărei variabile (ca de obicei)

Apelarea membrului unui variabile de tip structură se face prin operatorul (->) care dereferențiază automat.

```
Nume: Rubin
Nume stiintific: Corindon
Formula chimica: Al2O3
```

https://onlinegdb.com/ryG2wTvc4

Dacă pointerii se dereferențiază, se revine la operatorul (.)

pointers_to_struct_deref.c

```
#include <stdio.h>
#include <string.h>

struct Stone
   {
       char *nume;
       char *snume;
       char *formula;
       int duritate;
       float refr;
       char *foto;
       char *clr;
   };


int main()

{
   struct Stone *prubin, *psafir, *psmarald, *pdiamant, rubin, safir, smarald, diamant;

   prubin = &rubin; psafir = &safir; pdiamant = &diamant; psmarald = &smarald;

   (*prubin).nume =  "Rubin";
   (*prubin).snume = "Corindon";
   (*prubin).formula = "Al2O3";

   printf ("Nume: %s\n", (*prubin).nume);
   printf ("Nume stiintific: %s\n", (*prubin).snume);
   printf ("Formula chimica: %s\n", (*prubin).formula);
}
```

pointerii se definesc cu același tip de date (strucutra definită mai sus)

Pointerilor li se atribuie adresa fiecărei variabile (ca de obicei)

(*prubin) = rubin

```
Nume: Rubin
Nume stiintific: Corindon
Formula chimica: Al2O3
```

https://onlinegdb.com/HJJyOaP54

Structurile se pot grupa în vectori sau matrici

```
#include <stdio.h>
#include <string.h>

struct Stone
   {
       char *nume;
       char *snume;
       char *formula;
       int duritate;
       float refr;
       char *foto;
       char *clr;
   };


int main()

{
   struct Stone stones[4];

   stones[0].nume =  "Rubin";
   stones[0].snume = "Corindon";
   stones[0].formula = "Al2O3";

   printf ("Nume: %s\n", stones[0].nume);
   printf ("Nume stiintific: %s\n", stones[0].snume);
   printf ("Formula chimica: %s\n", stones[0].formula);
}
```

```
Nume: Rubin
Nume stiintific: Corindon
Formula chimica: Al2O3
```

arrays_of_structs.c

Șir de 4 variabile de tip Stone

Variabilele se adresează cu ajutorul index-ului șirului

https://onlinegdb.com/SkT-_TDcV

Typedef

Typedef este un operator care permite definirea unor alias-uri pentru tipurile de variabile.

typedefs.c

```c
#include <stdio.h>

typedef int asprete;
typedef char salau;
typedef float babusca;
typedef short int platica;

int main()
{
   asprete lng = 12213;
   salau cr = 'd';
   salau decl[] = "Nu stiu nimic";
   babusca srs = 12.6576;
   platica mm = 12;

   printf("lng: %d\n", lng);
   printf("cr: %c\n", cr);
   printf("decl: %s\n", decl);
   printf("srs: %f\n", srs);
   printf("mm: %d\n", mm);
   return 0;
}
```

https://onlinegdb.com/SJLduaP5N

```
lng: 12213
cr: d
decl: Nu stiu nimic
srs: 12.657600
mm: 12
```

typedef este extrem de utilizat, de pildă, în sursa nucleului linux (4.9.51) sunt 16338 de apariții

```c
typedef unsigned char bits8;
typedef signed char sbits8;
typedef unsigned short int bits16;
typedef signed short int sbits16;
typedef unsigned int bits32;
typedef signed int sbits32;
typedef unsigned long long int uint64;
typedef signed long long int int64;
```

Se folosește foarte frecvent pentru scurtarea numelui tipurilor de variabile

Typedef în definirea structurilor

typedef_struct.c

```c
#include <stdio.h>
#include <string.h>

typedef struct
   {
       char *nume;
       char *snume;
       char *formula;
       int duritate;
       float refr;
       char *foto;
       char *clr;
   } bolovan;


int main()

{
   bolovan stones[4];

   stones[0].nume =  "Rubin";
   stones[0].snume = "Corindon";
   stones[0].formula = "Al2O3";

   printf ("Nume: %s\n", stones[0].nume);
   printf ("Nume stiintific: %s\n", stones[0].snume);
   printf ("Formula chimica: %s\n", stones[0].formula);
}
```

utilizarea typedef împreună cu struct permite crearea aliasului bolovan pentru structura. Denumirea structurii nu mai este necesara.

Declararea variabilelor se face direct cu aliasul definit.

https://onlinegdb.com/Hk9w56DcE

Dimensiunea unei structuri

struct_dim_24.c

```c
#include <stdio.h>

typedef struct  {
   int a;  // 4 octeti
   int c;  //4 octeti
   double b;   // 8 octeti
   long long int d; // 8 octeti
} VARS;

int main() {
   printf("dim v1 %ld\n", sizeof(VARS) );
   return 0;
}
```

https://onlinegdb.com/B-zbTVcIT

```
dim v1 24
```

struct_dim_32.c

```c
#include <stdio.h>

typedef struct  {
   int a;  // 4 octeti
   double b;   // 8 octeti
   int c;  //4 octeti
   long long int d; // 8 octeti
} VARS;

int main() {
   printf("dim v1 %ld\n", sizeof(VARS) );
   return 0;
}
```

https://onlinegdb.com/lZpc9oL0h

```
dim v1 32
```

```c
int a;  // 4 octeti
int c;  //4 octeti
double b;   // 8 octeti
long long int d; // 8 octeti
```

4 + 4 + 8 + 8 = 24

Sigura diferență între cele două declarații ale structurii este faptul că au fost inversate variabilele c cu b.

4 + 8 + 4 + 8 = 32 ??

```c
int a;  // 4 octeti
double b;   // 8 octeti
int c;  //4 octeti
long long int d; // 8 octeti
```

Dimensiunea unei structuri

```
int a;   // 4 octeti
int c;   //4 octeti
double b;    // 8 octeti
long long int d; // 8 octeti
```

Structurile sunt stocate în memorie în zone continue, membrii lor fiind așezați unui după celălalt, în ordinea în care au fost declarați.

4 + 4 + 8 + 8 = 24

4 + 8 + 4 + 8 = 32 ??

```
int a;   // 4 octeti
double b;    // 8 octeti
int c;   //4 octeti
long long int d; // 8 octeti
```

Dimensiunea unei structuri (întotdeauna cu sizeof!)

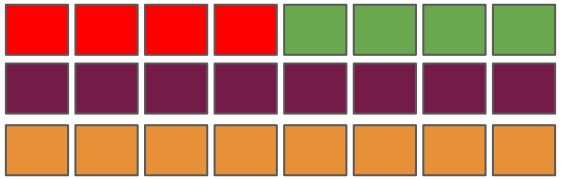
Chiar dacă memoria este adresabilă la nivel de octet, procesorul citește atâția octeți câți încap într-un registru (8 octeți în cazul unui procesor pe 64 de biți).

Compilatoarele aliniază variabilele astfel încât sa înceapă toate la cel mai din stânga octet. O variabilă nu poate fi citită în două citiri succesive. Din acest motiv, în funcție de ordinea declarării variabilelor, pot rămâne spații libere.

Pentru a încărca toată structura, în primul caz este nevoie de 24 de biți iar în al doilea de 32

Din acest motiv C nu permite comparația structurilor.

```
int a;   // 4 octeti
int c;   //4 octeti
double b;    // 8 octeti
long long int d; // 8 octeti
```



24

4 + 4 + 8 + 8 = 24

```
int a;   // 4 octeti
double b;    // 8 octeti
int c;   //4 octeti
long long int d; // 8 octeti
```

32

4 + 8 + 4 + 8 = 32

Copierea unei structuri în altă structură cu aceeași definiție

Există trei metode pentru a copia o structură definită și inițializată într-o altă structură cu aceeași definiție:

- Copierea prin atribuire.
- Copierea manuală, membru cu membru.
- Copierea memoriei ocupate de structură, octet cu octet, folosind funcția memcpy.

Atribuire

struct_copy_attr.c

```
#include <stdio.h>

typedef struct {
   char *start;
   char *end;
} mutare;

int main() {

   mutare m1, m2;
   m1.start = "E2";
   m1.end = "E4";
   printf ("M1.start = %s\n", m1.start);
   printf ("M1.end = %s\n", m1.end);
   m2 = m1;
   printf ("M2.start = %s\n", m2.start);
   printf ("M2.end = %s\n", m2.end);
}
```

https://onlinegdb.com/r1Sjq6wcE

```
M1.start = E2
M1.end = E4
M2.start = E2
M2.end = E4
```

membru cu membru

struct_copy_man.c

```
#include <stdio.h>

typedef struct mutare {
   char *start;
   char *end;
} mutare;

int main() {
   mutare m1, m2;
   m1.start = "E2";
   m1.end = "E4";
   printf ("M1.start = %s\n", m1.start);
   printf ("M1.end = %s\n", m1.end);
   m2.start = m1.start;
   m2.end = m1.end;
   printf ("M2.start = %s\n", m2.start);
   printf ("M2.end = %s\n", m2.end);
}
```

https://onlinegdb.com/ryCpcpvq4

```
M1.start = E2
M1.end = E4
M2.start = E2
M2.end = E4
```

Copierea structurilor cu memcpy

struct_memcpy.c

```c
#include <stdio.h>
#include <string.h>

typedef struct {
   char *start;
   char *end;
} mutare;


int main() {

   mutare m1, m2;
   m1.start = "E2";
   m1.end = "E4";
   printf ("M1.start = %s\n", m1.start);
   printf ("M1.end = %s\n", m1.end);

   memcpy (&m2, &m1, sizeof(mutare));

   printf ("M2.start = %s\n", m2.start);
   printf ("M2.end = %s\n", m2.end);

}
```

https://onlinegdb.com/SJYlipD94

void *memcpy(void *dest, const void *src, size_t n)

memcpy copiaza numărul de octeți din argumentul n începând de la adresa pointerului src, în spațiul echivalent de memorie care începe de la adresa pointerului dest.

Atenție la pointeri la copierea structurilor

struct_shallow_copy.c

```c
#include <stdio.h>
#include <string.h>

typedef struct {
   int *p1;
   int *p2;
} pointeri;

int main() {
   int fvar = 12;
   int *pfvar = &fvar;
   int svar = 123;
   int *psvar = &svar;

   pointeri POI1, POI2;

   POI1.p1 = pfvar;
   POI1.p2 = psvar;
   printf ("POI1.p1 = %d\n", *POI1.p1);
   printf ("POI1.p2 = %d\n", *POI1.p2);
   printf ("Ad POI1.p1 = %p\n", POI1.p1);
   printf ("Ad POI1.p2 = %p\n", POI1.p2);

   POI2 = POI1;
   fvar = 1921;

   printf ("POI1.p1 = %d\n", *POI1.p1);
   printf ("POI2.p1 = %d\n", *POI2.p1);
   printf ("POI2.p2 = %d\n", *POI2.p2);
   printf ("Ad POI2.p1 = %p\n", POI2.p1);
   printf ("Ad POI2.p2 = %p\n", POI2.p2);
}
```

https://onlinegdb.com/Sk7Xs6v5N

Shallow copy - se copiază pointerii dintr-o structură în alta iar la final ambele structuri vor trimite către aceleași date. Modificarea uneia dintre date (sau chiar a variabilei inițiale) determină modificarea datelor în ambele structuri.

Toate metodele de copiere prezentate fac shallow copy.

Deep copy - se aloca memorie, se dubleaza toate datele la care structura are acces, se definesc noi pointeri către noile zone de memorie.

Funcțiile pentru deep copy trebuie programate.

```
POI1.p1 = 12
POI1.p2 = 123
Ad POI1.p1 = 0x7fffb93d7f38
Ad POI1.p2 = 0x7fffb93d7f3c
POI1.p1 = 1921
POI2.p1 = 1921
POI2.p2 = 123
Ad POI2.p1 = 0x7fffb93d7f38
Ad POI2.p2 = 0x7fffb93d7f3c
```

Deep copy

struct_deep_copy.c

```c
#include <stdio.h>
#include <string.h>

typedef struct {
   int *p1;
   int *p2;
} pointeri;

int main() {
   int fvar = 12;
   int *pfvar = &fvar;
   int svar = 123;
   int *psvar = &svar;

   pointeri POI1, POI2;

   POI1.p1 = pfvar;
   POI1.p2 = psvar;
   printf ("POI1.p1 = %d\n", *POI1.p1);
   printf ("POI1.p2 = %d\n", *POI1.p2);
   printf ("Ad POI1.p1 = %p\n", POI1.p1);
   printf ("Ad POI1.p2 = %p\n", POI1.p2);

   POI2 = POI1;
   fvar = 1921;

   printf ("POI1.p1 = %d\n", *POI1.p1);
   printf ("POI2.p1 = %d\n", *POI2.p1);
   printf ("POI2.p2 = %d\n", *POI2.p2);
   printf ("Ad POI2.p1 = %p\n", POI2.p1);
   printf ("Ad POI2.p2 = %p\n", POI2.p2);
}
```

În funcție de complexitatea structurilor (care poate fi foarte mare) funcțiile de deep copy pot deveni extrem de complicate.

1. Se alocă un spațiu corespunzător de memorie pentru p1
2. Se alocă un spațiu corespunzător de memorie pentru p2
3. Se atribuie valoarea dereferentiată a POI1.p1 adresei către care trimite POI2.p1
4. Se atribuie valoarea dereferentiată a POI1.p2 adresei către care trimite POI2.p2

```c
POI2.p1 = (malloc(sizeof(int)));
POI2.p2 = (malloc(sizeof(int)));
*POI2.p1 = *POI1.p1;
*POI2.p2 = *POI1.p2;
```

În exemplul dat, fiind vorba de numere întregi, se putea evita utilizarea malloc, este dat ca exemplu doar.

Union, Enum

Union

O uniune este un tip compus de dată în C care poate fi definită cu mai mulți posibili membrii (similar unei structuri) dar care nu poate conține decât o singură valoare (doar unul dintre membrii este definit), pentru că datele sunt stocate în aceeași locație de memorie.

```
#include <stdio.h>
#include <string.h>


union Data {
  int i;
  float f;
  char str[20];
};


int main()  {

   union Data rezultat;
   printf ("Memoria ocupata: %ld\n", sizeof(rezultat));
   rezultat.i = 21;
   printf ("Rezultat i: %d\n", rezultat.i);
   rezultat.f = 21.232;
   printf ("Rezultat f: %f\n", rezultat.f);
   strcpy(rezultat.str, "Nu stiu ce sa scriu");
   printf ("Rezultat str: %s\n", rezultat.str);
   printf ("Rezultat i: %d\n", rezultat.i);
   return 0;
}
```

```
Memoria ocupata: 20
Rezultat i: 21
Rezultat f: 21.232000
Rezultat str: Nu stiu ce sa scriu
Rezultat i: 1931507022
```

Memoria ocupată este egală cu memoria ocupată de cel mai voluminos membru

După atribuirea valorii “Nu stiu ce sa scriu” membrului de tip șir de caractere, nu se mai poate accesa nici unul din membrii anteriori (valorile sunt greșite)

union.c

Union este util, în special ca membru al unor structuri mai complexe, în cazuri în care datele reale nu se supun tipurilor C - de exemplu așteptăm numere dar nu știm dacă vor fi întregi lungi sau scurte, float-uri, etc.

https://onlinegdb.com/ryncIsRFN

Union peste struct

```c
#include <stdio.h>


typedef union
{
 struct
 {
   unsigned char bit8 : 1;
   unsigned char bit7 : 1;
   unsigned char bit6 : 1;
   unsigned char bit5 : 1;
   unsigned char bit4 : 1;
   unsigned char bit3 : 1;
   unsigned char bit2 : 1;
   unsigned char bit1 : 1;
 }u;
 char status;
}DeviceStatus;


int main()
{
   DeviceStatus ds;
   ds.status = 121;
   printf("bits: %d %d %d %d %d %d %d %d
",ds.u.bit1,ds.u.bit2,ds.u.bit3,ds.u.bit4,ds.u.bit5,ds.u.bit6,ds.u.bit7,ds.u.bit8 );
}
```

char status și structura numită u împart aceeași memorie. Astfel, dacă inițializăm ds.status cu o valoare corespunzătoare (121), aceiași biți utilizați pentru stocarea variabilei de tip caracter “status” pot fi accesați de structura u care are 8 membri a câte un bit.

Această tehnică se folosește foarte mult în programarea registrelor hardware.

Tehnica se bazează pe faptul că în C putem specifica dimensiunea în biți a membrilor structurilor și a uniunilor. Aceasta se face prin folosirea semnului :

unsigned char bit8 : 1 - definește folosirea unui singur bit

Acest mod de definire se numește “bit field”.

Această tehnică este extrem de utilizată în “despachetarea” rapidă a multor date în biți sau în fragmente de alte dimensiuni (registre hardware, culori, adrese ip, etc.). în sursa nucleului Linux cuvântul union este prezent de 3032 de ori

```
bits: 0 1 1 1 1 0 0 1
```

https://onlinegdb.com/SkHSL3AYV

union_struct.c

Enum

enum este un tip de date care crează un alias între cuvinte și numere întregi. Este folosit în special pentru a face codul sursă mai ușor de citit.

Enum înlocuiește numerele cu cuvintele.

```c
#include <stdio.h>

enum year{Jan, Feb, Mar, Apr, May, Jun,
Jul, Aug, Sep, Oct, Nov, Dec};

int main()
{
  int i;
  enum year ch;
  ch = May;
  for (i=Jan; i<=Dec; i++) {
     printf("%d ", i);
  }
  printf ("\nch este: %d\n",ch);
  return 0;
}
```

enum_1.c

```
0 1 2 3 4 5 6 7 8 9 10 11
ch este: 4
```

```c
#include <stdio.h>

enum year{Jan=10, Feb, Mar, Apr, May,
Jun, Jul, Aug, Sep, Oct, Nov, Dec};

int main()
{
  int i;
  enum year ch;
  ch = May;
  for (i=Jan; i<=Dec; i++) {
     printf("%d ", i);
  }
  printf ("\nch este: %d\n",ch);
  return 0;
}
```

enum_init.c

Etichetele din enum pot fi inițializate parțial sau total.

```
10 11 12 13 14 15 16 17 18 19 20 21
ch este: 14
```

https://onlinegdb.com/ryYIc30KE

WOULD YOU LIKE TO KNOW MORE?

Plain Text - Dylan Beattie - NDC Oslo 2021

Reprezentarea textului - Morse, ASCII, UTF, etc., erori celebre determinate de codificarea textului și de ce are Billy Joel un album numit "Kohuept"

https://www.youtube.com/watch?v=_mZBa3sqTrI

Unicode and Byte Order

Reprezentarea textului - despre diferitele tipuri de codificare a caracterelor și influența ordinii de stocare a octeților.

https://www.youtube.com/watch?v=bbkUn0o3L1Y

David Patterson: Computer Architecture and Data Storage | Lex Fridman Podcast #104

David Patterson este unul dintre inventatorii arhitecturilor de tip RISC (probabil cel care a inventat termenul) și al arhitecturii de stocare de tip RAID.

https://www.youtube.com/watch?v=naed4C4hfAg

Programming Paradigms (Stanford)

Cursul de programare de la Stanford

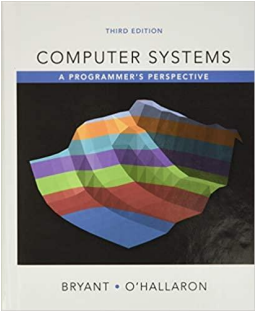
https://www.youtube.com/watch?v=Ps8jOj7diA0&list=PLD28639E2FFC4B86A

Stop Writing Dead Programs by Jack Rusher (Strange Loop 2022)

Cât de mult din obiceiurile noastre de astăzi vin din menținerea obiceiurilor vechi

https://www.youtube.com/watch?v=8Ab3ArE8W3s

WOULD YOU LIKE TO KNOW MORE?



Computer Systems: A Programmer's Perspective
Randal Bryant, David O'Hallaron

ISBN: 978-0134092669

GitHub - shihyu/CSAPP2e

21:Everything u need 2 know about pointers -Richard Buckland

Pointeri, referințe - o lecție la tablă

https://www.youtube.com/watch?v=Rxvv9krECNw

5. C to Assembly

Detalii despre cum este tradus codul C în limbajul de asamblare. Se vorbeste si despre LLVM - un compilator modern care poate compila mai multe limbaje de programare

https://www.youtube.com/watch?v=wt7a5BOztuM

The Next Decade of Software Development - Richard Campbell - NDC Oslo 2021

O incercare de a imagina viitorul imediat al dezvoltarii software pe baza celor intamplate pana acum, împreună cu ceva sfaturi cu privire la modul în care ar trebui abordată o carieră în programare.

https://www.youtube.com/watch?v=a_LM3FJozr0

That's all Folks!